# Lesson 2 — Introduction to Cloud Computing

**INTE 22253 | Source: Lecture 02 "Introduction to Cloud Computing: From Basics to Modern Developments" (45 slides) | Builds on Lesson 1 (a cloud is a very large distributed system)**

**WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON**

- Explain why the data explosion (big data, 5 V's) drives cloud computing, using the electricity analogy.
- Give the definition of cloud computing and the **NIST five essential characteristics**.
- Explain **IaaS, PaaS and SaaS** (who manages which layer, examples, best use) and the other "as a service" models.
- Compare **public, private, hybrid, community and multi-cloud** deployment models.
- Explain the enabling technologies, especially **virtualization**, and the cloud characteristics.
- Describe modern developments: containers, Kubernetes, serverless, edge and fog computing, AI in the cloud, the **shared responsibility model**, **Zero Trust**, FinOps, compliance, green cloud, multi-cloud and DevOps.

## 1. The world of data (slides 3–6)

Almost everything we do creates digital data: documents and files, media (photos, videos, music), communications (email, chat, video calls), personal data (calendar, contacts, location), health data (wearables, medical records) and financial data (banking, transactions). Scale on the slides: **2.5 quintillion bytes created every day**, 5B+ Internet users (2024), 500M+ tweets a day, 720K hours of YouTube video uploaded a day.

**BIG DATA (DEFINITION)**
Large pools of data that can be captured, communicated, aggregated, stored and analysed to deliver insights and value.

| The 5 V's | Meaning | Slide example |
|---|---|---|
| **Volume** | The sheer amount of data | 1.2 ZB (2010) → 120+ ZB (2024). 1 ZB (zettabyte) = $10^{21}$ bytes |
| **Velocity** | The speed at which data arrives and must be processed | Real-time streaming and processing |
| **Variety** | The different forms of data | Structured (tables), semi-structured (JSON, XML), unstructured (images, video, text) |
| **Veracity** | How trustworthy/accurate the data is | Data quality and accuracy |
| **Value** | The useful result we get from it | Business insights and decisions |

**What we do with data** (slide 6): **store** (databases, file systems, object storage), **secure** (encryption, access control, compliance), **share** (APIs, collaboration platforms), **process** (real-time analytics, batch processing), **analyze** (ML, AI, BI dashboards), **stream** (IoT sensors, live pipelines). A single company cannot cheaply buy enough machines for all this — this is what pushes computing into the cloud.

## 2. From product to service: the electricity analogy (slides 7–8)

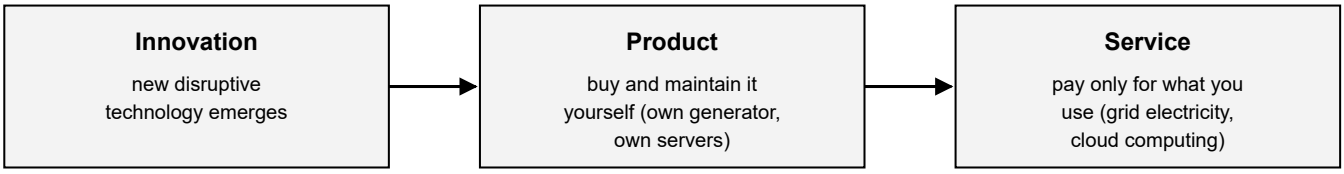



Figure 1 — Electricity, water and banking moved from product to service. Computing follows the same path.

| Stage | Electricity | Computing |
|---|---|---|
| Then | Companies ran private generators: expensive, unreliable, needed specialists | Companies bought and ran their own servers in their own server rooms |
| Transition | Electricity became a shared utility from the grid; pay for what you use | Large providers built huge shared data centres |
| Now | Plug in and pay per unit | **Cloud**: computing on demand, no upfront servers, pay per use, managed by experts |

## 3. What is cloud computing? (slides 9–12)

**DEFINITION (SLIDE 9)**
"Cloud computing is the delivery of computing **as a service rather than a product**, whereby shared resources, software and information are provided to computers and other devices as a **metered service over a network**." Short form: **on-demand IT services on a pay-as-you-go basis over the Internet**.

**Key concepts (slide 10):**

- **Remotely hosted**: services and data are stored and run on remote infrastructure, not on local machines.
- **Ubiquitous access**: accessible from anywhere, anytime, on any device.
- **Commodified computing**: computing as a utility like water or electricity; pay only for what you consume.
- **Multi-tenancy**: multiple users (**tenants**) share the same physical infrastructure securely and independently, like flats in one apartment block.

### NIST's five essential characteristics (slide 11) — very likely exam question

NIST (the US National Institute of Standards and Technology) wrote the standard definition. A service is "cloud" only if it has all five.

| # | Characteristic | Meaning | Example |
|---|---|---|---|
| 1 | **On-demand self-service** | Users provision computing resources automatically, **without human interaction** from the provider. | You click "Launch instance" and get a server in a minute; nobody at AWS approves it. |
| 2 | **Broad network access** | Available over the network through standard mechanisms (phones, tablets, laptops). | Open Google Drive from a phone or a laptop browser. |
| 3 | **Resource pooling** | The provider's resources are pooled to serve many consumers using a **multi-tenant** model; the customer does not know the exact physical location. | Your VM and another company's VM run on the same physical server. |
| 4 | **Rapid elasticity** | Capabilities can be provisioned and released quickly, even automatically, to scale with demand. | An online shop goes from 2 to 20 servers during a sale, then back to 2. |
| 5 | **Measured service** | Resource use is monitored, controlled and reported — pay for what you use. | Bill = CPU-hours + GB stored + GB transferred. |

Memory aid: **"On Broad Pools, Rapid Meters"** (On-demand, Broad, Pooling, Rapid, Measured).

### Why cloud computing? (slide 12)

| Benefit | Explanation |
|---|---|
| Pay-as-you-go | No upfront **capital expenditure (CapEx)** on hardware; costs become **operating expenditure (OpEx)**. Reduces **Total Cost of Ownership (TCO)** = all costs of owning (hardware, power, cooling, staff, space). |
| Scale instantly | Rent and release resources as needed. |
| Simplified IT | The provider manages the infrastructure; you need only Internet access. |
| Flexible options | Choose software, OS and platforms; access from anywhere. |
| Resource efficiency | Sharing reduces idle resources; better CPU/storage utilisation. |
| Green computing | Sharing = fewer servers, less power, lower carbon footprint. |

### Worked example: own server vs pay-as-you-go (illustrative numbers)

A student project needs a server for a 2-week exam-results peak each semester (2 semesters a year), and almost nothing the rest of the year.

| Option | Calculation | Yearly cost |
|---|---|---|
| Buy a server (sized for the peak) | Rs. 600,000 hardware spread over 3 years = 200,000/yr, + power/maintenance 50,000/yr | **Rs. 250,000**, mostly idle |
| Cloud VM, pay per hour | 4 weeks × 7 days × 24 h = 672 h × Rs. 100/h | **Rs. 67,200**, nothing paid when idle |

The point (not the exact numbers): for spiky or uncertain demand, measured service plus elasticity is much cheaper than owning capacity for the peak.

## 4. Cloud service models (slides 13–17)

A **service model** says **how much of the computing stack the provider manages for you**. The stack (bottom to top): networking, storage, servers, virtualization, operating system (OS), middleware, runtime, data, applications.

- **Middleware**: software between the OS and applications (web servers, message queues, database connectors).
- **Runtime**: the environment the code runs in (Java JVM, Python interpreter, Node.js).

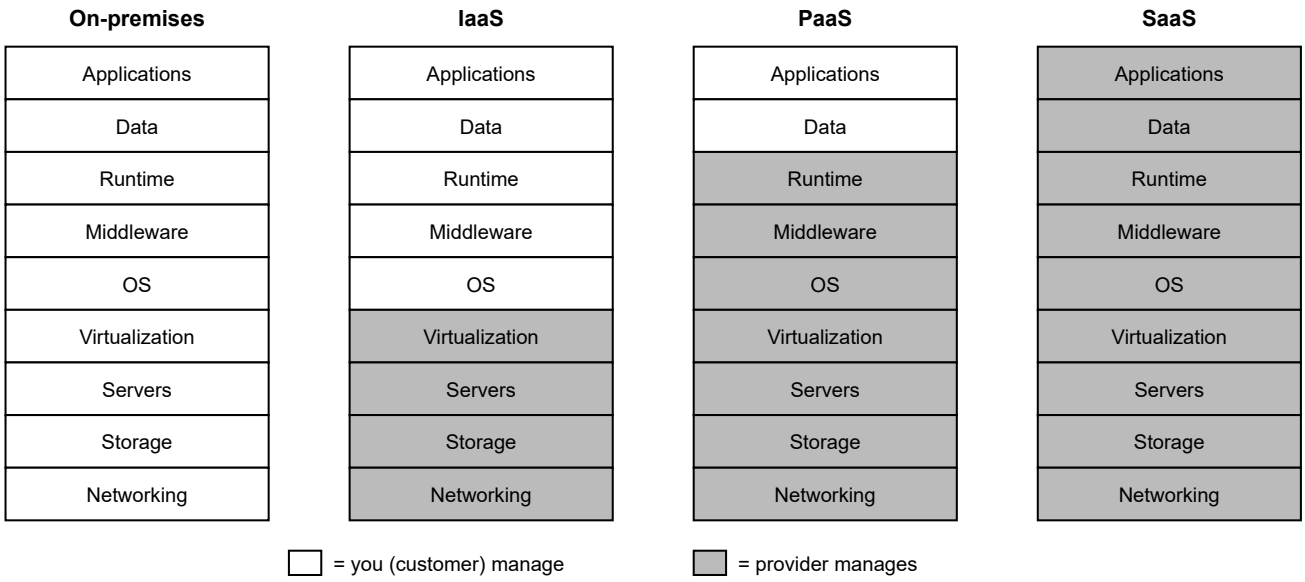



Figure 2 — Who manages what. Moving right, the provider takes over more layers and you control less (slide 14 shows the IaaS split).

| Model | What you get | You manage | Examples (slides) | Best for |
|---|---|---|---|---|
| **IaaS** Infrastructure as a Service | Virtualized computing infrastructure: VMs, storage, networking | OS and everything above it (OS, middleware, runtime, data, apps) | AWS EC2, Azure VMs, Google Compute Engine | Lift-and-shift migrations (moving existing apps unchanged), dev/test environments, HPC workloads |
| **PaaS** Platform as a Service | A platform to build and deploy apps; developers focus only on writing code | Only the application (and its data) | Heroku, Google App Engine, AWS Elastic Beanstalk, Azure App Service, Firebase | Developers who want to ship code without managing servers |
| **SaaS** Software as a Service | Complete, ready-to-use software over the Internet; no installation; pay-as-you-go/subscription | Nothing technical — just your data and settings | Gmail, Salesforce, Microsoft 365, Zoom | End users who just need the application |

**PaaS features (slide 15):** auto-scaling (handles traffic spikes), CI/CD pipelines (automated build, test, deploy), managed databases (no DBA; provider handles backups), dev frameworks (pre-built runtimes), built-in networking (load balancers, SSL certificates), monitoring (built-in logging and performance tools).

**SaaS examples (slide 16):** email (Gmail, Outlook 365), video conferencing (Zoom, Teams, Meet), CRM (Salesforce, HubSpot), office suites (Microsoft 365, Google Workspace), media (Spotify, Netflix), security (Okta, CrowdStrike), analytics (Tableau Online, Power BI), collaboration (Slack, Notion, Asana).

**PIZZA ANALOGY (HELPS IN EXPLANATIONS)**

On-premises = make pizza at home from scratch. IaaS = buy a ready base and oven time; you add toppings and bake. PaaS = pizza delivered; you just provide the table and drinks. SaaS = eat at a restaurant; everything is done for you.

### Beyond IaaS/PaaS/SaaS: "Everything as a Service" (slide 17)

| Model | Meaning | Examples |
|---|---|---|
| **FaaS** Function as a Service | Run individual functions on demand (serverless, section 8.3) | AWS Lambda, Azure Functions |
| **DBaaS** Database as a Service | Managed databases — no DBA needed | AWS RDS, Azure SQL, MongoDB Atlas |
| **STaaS** Storage as a Service | Scalable object/file storage | S3, Azure Blob, Google Cloud Storage |
| **AIaaS** AI as a Service | Pre-trained ML models via an API | AWS SageMaker, Azure AI, Google Vertex |
| **NaaS** Network as a Service | Virtual networking on demand, using Software-Defined Networking (SDN, Lesson 3) | SDN-based virtual networks |
| **SECaaS** Security as a Service | Cloud-based security: SIEM (security event monitoring), CASB (cloud access security broker), DLP (data loss prevention) | CrowdStrike, Zscaler |

### Worked example: pick the service model

| Scenario | Model | Why |
|---|---|---|
| A company moves its existing ERP server unchanged to the cloud and needs full control of the OS | IaaS | Lift-and-shift; they manage OS and up |
| A startup writes a Python web app and does not want to patch servers | PaaS | Only the code is theirs; scaling and runtime managed |
| A school needs email and video meetings for staff | SaaS | Ready-made software (Microsoft 365, Zoom) |
| Resize every uploaded photo once, when it arrives | FaaS | Short, event-triggered, pay per call |

## 5. Cloud deployment models (slide 18)

A **deployment model** says **who owns the infrastructure and who can use it** (a different question from the service model).

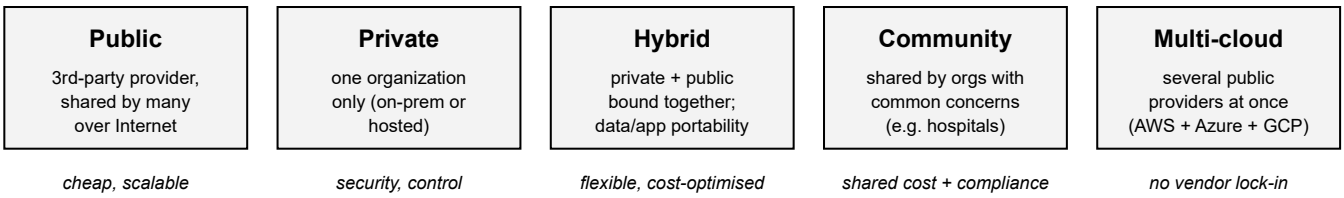



Figure 3 — Deployment models. Community cloud is named in Lecture 01 (slide 7); Lecture 02 adds multi-cloud.

| Model | Definition | Pros | Cons | Examples |
|---|---|---|---|---|
| **Public** | Owned and operated by a third-party provider, delivered over the Internet, shared by many customers | Low cost, high scalability, no maintenance | Less control; data on someone else's hardware; compliance concerns | AWS, Azure, Google Cloud |
| **Private** | Cloud infrastructure operated exclusively for a single organization | High security, compliance, control | Expensive; the organization pays for and runs capacity | VMware, OpenStack (software to build one) |
| **Hybrid** | Combination of public and private clouds, bound together by technology enabling data and application portability | Flexibility, cost-optimised (sensitive data private, peaks to public = **cloud bursting**) | More complex to integrate and secure | Sri Lankan banks (Commercial Bank, HNB) for core banking |
| **Community** *[L01]* | Shared by several organizations with common requirements (security, policy, compliance) | Costs shared; meets sector rules | Less flexible than public | Government agencies, hospitals, universities |
| **Multi-cloud** | Using multiple public cloud providers simultaneously | Avoid vendor lock-in, best-of-breed services | More skills and management needed | 2024: used by 87% of enterprises |

**Hybrid vs multi-cloud** (common confusion): hybrid = private + public; multi-cloud = more than one *public* provider. An organization can be both.

## 6. Enabling technologies and characteristics (slides 19–21)

| Enabling technology | How it makes cloud possible |
|---|---|
| Internet & Web 2.0 | High-bandwidth, always-on connectivity makes remote computing feasible. |
| **Virtualization** | Divides one physical machine into many virtual machines, maximising utilisation (the core enabler). |
| Parallel & distributed computing | Splitting workloads across many machines enables massive scale (Lesson 1). |
| Utility computing | Pay-per-use billing that mirrors traditional utilities. |
| Storage technologies | Object storage and distributed file systems (HDFS, GCS) enable petabyte-scale data (Lesson 7). |
| Fault tolerance | Redundancy and replication keep services up when components fail. |

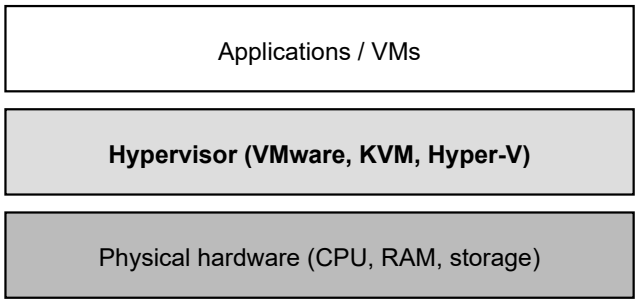



Figure 4 — Virtualization stack (slide 20).

**Virtualization: the core enabler.** A **hypervisor** is software that sits on the hardware and creates **virtual machines (VMs)**, each acting like a separate computer with its own OS. Benefits on the slide:

- **Server consolidation**: many lightly-used servers become VMs on fewer physical machines.
- **Isolation** between workloads: a crash in one VM does not affect others.
- **Rapid VM provisioning**: minutes, not weeks.
- **Snapshotting & migration**: save a VM's state; move it to another host.

Details: Lessons 5 and 8.

**Cloud computing characteristics (slide 21): massive scale** (global data centres, millions of servers); **homogeneity** (standardised hardware for consistent management); **resilient computing** (auto-healing recovery);

**geographic distribution** (regions for low latency and compliance); **low-cost software** (open-source stacks); **advanced security** (shared responsibility model); **service orientation** (everything exposed as an API — composable, interoperable); **virtualization** (physical resources abstracted into flexible logical ones).

**Market (slides 22–23):** AWS 31%, Azure 25%, Google Cloud 11%, Alibaba 4% (Synergy Research, Q3 2024). Public cloud revenue grew from $242B (2019) to an estimated $679B (2024) — about 3× in 5 years — and is expected to exceed $1 trillion by 2027.

## 7. Modern cloud developments (2020–2025): overview (slide 24)

Six trends: containers & Kubernetes · serverless computing · edge & fog computing · AI/ML as a service · Zero Trust security · sustainability & green cloud.

### 7.1 Containers and Docker (slide 25)

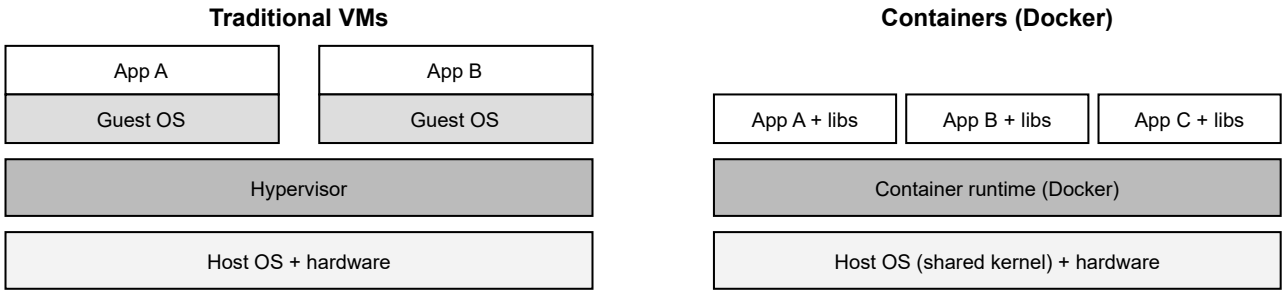



Figure 5 — VMs vs containers (slide 25). Containers are faster, lighter and more portable than VMs.

A **container** packages an application with its libraries and settings, but **not** a whole OS: all containers on a host share the host's OS **kernel** (the core of the OS that controls hardware). **Docker** is the most popular tool to build and run containers. Lesson 3 (Docker) and Lesson 5 (VM vs container) go deeper.

### 7.2 Kubernetes: container orchestration (slide 26)

With hundreds of containers on many machines, someone must decide where each runs, restart failed ones and spread traffic. **Kubernetes (K8s)** automates the **deployment, scaling and management of containerized applications**. Six features:

| Feature | What it does |
|---|---|
| Auto-scaling | Scales Pods (groups of containers) up/down based on CPU, memory or custom metrics |
| Self-healing | Restarts failed containers; kills containers that fail health checks |
| Load balancing | Distributes network traffic to Pods for a stable deployment |
| Rolling updates | Deploys new versions with zero downtime; instant rollback |
| Service discovery | Automatically assigns DNS names and IP addresses to containers |
| Storage orchestration | Automatically mounts storage — local, cloud or network |

Managed Kubernetes: **Amazon EKS, Azure AKS, Google GKE**. Full detail in Lesson 6.

### 7.3 Serverless computing (FaaS) (slide 27)

> **KEY SENTENCE**
> "Serverless" does **not** mean no servers — it means **developers don't manage servers**. The provider provisions, scales and manages infrastructure automatically.

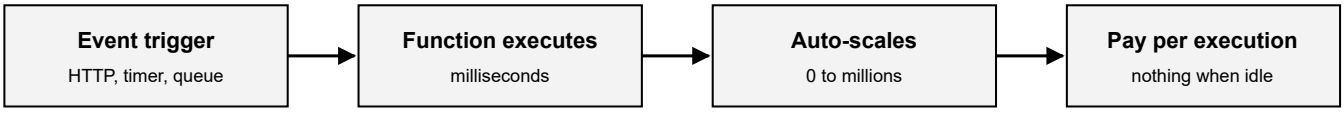



Figure 6 — Serverless flow (slide 27).

### Advantages

- No server management
- Auto-scales, including to zero
- Pay only per invocation
- Fast deployment

### Disadvantages

- **Cold start latency**: the first call after idle is slow because the provider must start a new environment
- Debugging is harder (short-lived, distributed functions)

Examples: AWS Lambda, Azure Functions, Google Cloud Functions, Cloudflare Workers. **Serverless vs containers** (knowledge check): serverless for event-driven, infrequent, short-lived functions; containers for long-running, stateful, complex apps.

## 7.4 Edge, fog and cloud computing (slides 28–29)

**Edge computing** = processing data **close to where it is generated** (at the "edge" of the network) instead of in a central data centre. Reason: **latency** — sending data to a far data centre and waiting for the answer takes too long for some uses.

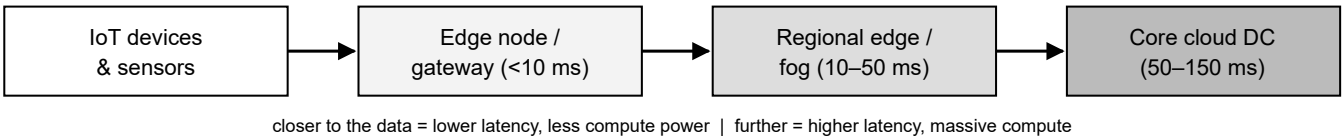



Figure 7 — Edge → fog → cloud tiers (slides 28–29).

| Aspect | Cloud computing | Fog computing | Edge computing |
|---|---|---|---|
| Where | Centralized data centres | Between edge and cloud; aggregates edge data | On the device or a nearby gateway; processes at the source |
| Latency | High (50–150 ms) | Medium (10–50 ms) | Ultra-low (<10 ms) |
| Compute | Massive | Moderate | Lightweight, real-time |
| Ideal for | Batch analytics, storage | Aggregation, local analytics; Cisco's term for distributed compute | Instant decisions |
| Examples | AWS, Azure, GCP | OpenFog, Cisco IOx | AWS Wavelength, Azure Edge |

**Edge use cases:** autonomous vehicles (real-time decisions cannot wait for a cloud round-trip), industrial IoT (sensors control machines locally), remote healthcare (devices process vitals for instant alerts), cloud gaming (ultra-low latency). **MEC** (in the slide title) = Multi-access Edge Computing: edge servers placed in the mobile/telecom network.

## 7.5 AI and machine learning in the cloud (slides 30–31)

Training large AI models needs thousands of GPUs, which only the cloud can supply at that scale: "the cloud is the engine of the AI revolution". Platforms: AWS SageMaker (build, train, deploy), Azure AI / OpenAI (GPT-4, DALL-E, Cognitive Services), Google Vertex AI (AutoML, model garden, Gemini); NVIDIA A100/H100 GPU instances. Generative AI cloud spending is projected to exceed $200B a year by 2028.

GenAI and cloud are **symbiotic** (each helps the other): foundation models (GPT-4, Claude, Gemini, LLaMA) need cloud GPUs; providers built AI chips (TPUs, Trainium, Inferentia); AI is offered through simple REST APIs; companies fine-tune models on private data; VPCs and private endpoints protect sensitive training data; new vector databases (Pinecone, Weaviate, pgvector) power AI apps.

# 8. Cloud security (slides 32–33)

### 8.1 The shared responsibility model

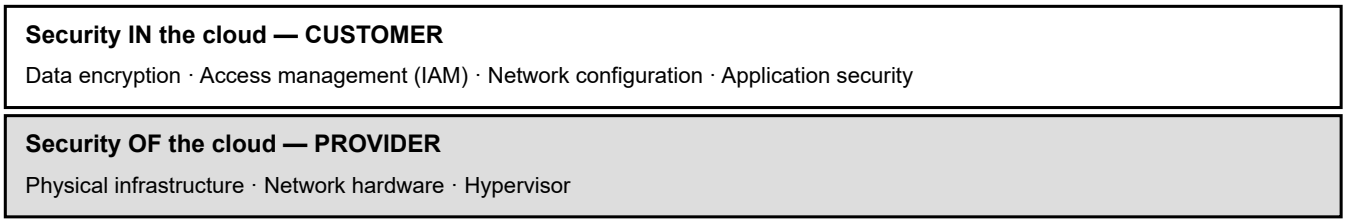



Figure 8 — Shared responsibility (slide 32). The provider secures the cloud itself; the customer secures what they put in it.

Example: if a company leaves an S3 bucket open to the public, that is the **customer's** fault (configuration "in" the cloud), not AWS's. **IAM** = Identity and Access Management: users, roles and permissions.

### 8.2 Zero Trust security

Traditional security trusted everything *inside* the company network (a "castle and moat" **perimeter**). In the cloud there is no clear perimeter, so this is obsolete. **Zero Trust: "never trust, always verify"** — assume a breach and verify continuously.

| Principle | Meaning |
|---|---|
| **Verify explicitly** | Authenticate and authorize using all data points: identity, location, device health, workload |
| **Least privilege** | Give only the access needed, at the right time, for the right purpose |
| **Assume breach** | Design as if attackers are already inside; minimise the **blast radius** (how much damage one breach can do) and segment access |

Tools: Microsoft Entra ID, Zscaler, CrowdStrike, Palo Alto Networks SASE.

# 9. Cost, compliance, sustainability, strategy (slides 34–38)

### 9.1 FinOps: cloud cost optimisation

**FinOps** = bringing financial accountability to cloud spending (costs can spiral without management).

| Technique | Meaning |
|---|---|
| Right-sizing | Match instance size to the real workload; oversized instances waste money |
| Reserved instances | Commit for 1–3 years for predictable workloads; save up to 72% |
| Spot / preemptible VMs | Use spare capacity at 60–90% discount, for fault-tolerant work (the provider can take it back) |
| Auto-scaling | Scale down at low demand; don't pay for idle resources overnight/weekends |
| Storage tiering | Move rarely used data to cheaper tiers (Glacier, Archive) |
| Cost allocation tags | Tag every resource to track spending by team, project or environment |

**Worked example:** a VM costs $100/month on demand. Reserved (up to 72% off) ≈ $28/month; spot (60–90% off) ≈ $10–40/month but can be interrupted. A web server that must always run → reserved; a batch video-rendering job that can restart → spot.

### 9.2 Compliance, governance and data sovereignty

**Data sovereignty** = data is subject to the laws of the country where it is stored, so some data must stay in-country.

| Standard | Scope |
|---|---|
| GDPR | EU General Data Protection Regulation: strict personal-data rules, right to erasure, data residency |
| HIPAA | US health data: must be encrypted and access-logged |
| PCI-DSS | Payment Card Industry standard for systems processing card data |
| ISO 27001 | International information-security management standard |
| SOC 2 | Service Organization Controls; providers get Type I/II attestations |
| PDPA / DPDPA | Personal Data Protection Acts in Sri Lanka, India and South-East Asia |

### 9.3 Sustainability and green cloud (slide 36)

Data centres use about **1–2% of global electricity**. Google: carbon neutral since 2007, targeting 24/7 carbon-free energy by 2030. Microsoft: carbon negative and water positive by 2030, 100% renewable. AWS: 100% renewable by 2025, net-zero carbon by 2040. Shared cloud infrastructure is **65–80% more energy-efficient** than individual on-premises servers.

### 9.4 Multi-cloud and hybrid strategy (slide 37)

87% of enterprises use several providers. Five reasons: (1) **avoid vendor lock-in** (not dependent on one provider's pricing, features or outages); (2) **best-of-breed** (AWS for compute, GCP for AI/ML, Azure for Microsoft integration); (3) **regulatory compliance** (local provider for in-country data); (4) **cost arbitrage** (use the cheapest per workload); (5) **redundancy** (fail over to another provider during an outage). Tools: Terraform, Pulumi, Kubernetes (cloud-agnostic), HashiCorp Vault.

### 9.5 DevOps and cloud-native development (slide 38)

**Cloud-native** apps are designed from the ground up to exploit the cloud.

| Practice | Meaning |
|---|---|
| Microservices | Split the app into small independent services; each scales independently |
| CI/CD pipelines | Continuous Integration / Continuous Delivery: automated build, test and deploy; release many times a day safely |
| Infrastructure as Code | Manage infrastructure with code files (Terraform, CloudFormation): version-controlled, repeatable |
| Observability | Logs, metrics, traces to understand production behaviour (Datadog, Grafana) |
| Chaos engineering | Deliberately inject failures to test resilience (Netflix Chaos Monkey) |
| GitOps | Git is the single source of truth for infrastructure; the cluster auto-syncs to Git |

## 10. Case studies, Sri Lanka, future, careers (slides 39–42)

| Case | Cloud | Fact from the slide |
|---|---|---|
| Netflix | AWS | Moved from own data centres; 260M+ subscribers; 100K+ instances at peak |
| Airbnb | AWS | Startup to $75B valuation; 150M+ listings; no data-centre costs |
| Spotify | Google Cloud | BigQuery analytics; 600 GB/day for recommendations |
| BMW | AWS + Azure | Multi-cloud; 3M+ connected cars sending real-time telemetry |

**Sri Lanka & South Asia:** ICTA cloud-first policy and Lanka Government Cloud; banks (Commercial Bank, HNB) on hybrid cloud; Dialog and SLT offer cloud to SMEs; universities on Microsoft 365/Google Workspace; AWS/Azure regions in Singapore and India give sub-10 ms latency; startups/fintech scale on AWS/GCP.

**Future trends:** quantum computing in the cloud (2025–30; threatens today's encryption, so quantum-resistant crypto is needed); AI-Ops / autonomous cloud (AI auto-heals, predicts scaling); **confidential computing** (data processed while still encrypted); 6G edge-to-cloud; neuromorphic (brain-inspired) chips such as Intel Loihi; space-based cloud (Starlink, Kuiper, Azure Orbital). **Certifications:**

AWS Cloud Practitioner → Solutions Architect / Developer → DevOps Professional; Azure AZ-900, AZ-104, AZ-204, AZ-305; Google Cloud Digital Leader, Associate Cloud Engineer, Professional Cloud Architect / ML Engineer.

## 11. Lecture summary (slide 44)

1. Data explosion drives demand: 2.5 quintillion bytes a day; the cloud stores, processes and analyses it.
2. Cloud = IT as a utility: product → on-demand, metered service.
3. Three service models (IaaS, PaaS, SaaS) plus FaaS, DBaaS, AIaaS and more.
4. Deployment models: public, private, hybrid, multi-cloud — choose by cost, control and compliance.
5. Modern developments: containers, Kubernetes, serverless, edge computing, AI/ML cloud, Zero Trust.
6. Sustainability matters: consolidation cuts carbon; providers target net-zero.

## 12. Exam-style questions with model answers

**Q1. Define cloud computing and list the five essential characteristics defined by NIST. (6 marks)**
Cloud computing is the delivery of computing as a service rather than a product, where shared resources, software and information are provided to devices as a metered service over a network (on-demand IT on a pay-as-you-go basis). NIST characteristics: on-demand self-service (provision without human interaction), broad network access (standard devices over the network), resource pooling (multi-tenant shared resources), rapid elasticity (scale out and in quickly), measured service (usage monitored and billed).

**Q2. Compare IaaS, PaaS and SaaS in terms of what the customer manages, with one example each. (6)**
IaaS: provider manages networking, storage, servers and virtualization; the customer manages the OS, middleware, runtime, data and applications (AWS EC2). PaaS: the provider also manages OS, middleware and runtime; the customer manages only the application and data (Google App Engine). SaaS: the provider manages the whole stack; the customer only uses the software (Gmail, Microsoft 365).

**Q3. Distinguish public, private and hybrid clouds and state one advantage of each. (6)**
Public: owned by a third party and shared over the Internet — low cost and high scalability. Private: operated exclusively for one organization — high security, control and compliance. Hybrid: public and private bound together with data/app portability — flexibility and cost optimisation (keep sensitive data private, burst peaks to public).

**Q4. Explain the shared responsibility model with an example. (4)**
Security is split: the provider is responsible for security OF the cloud (physical infrastructure, network hardware, hypervisor); the customer is responsible for security IN the cloud (data encryption, IAM, network configuration, application security). E.g. if a customer misconfigures an S3 bucket as public and data leaks, it is the customer's responsibility.

**Q5. "Serverless means there are no servers." Discuss. Give two advantages and two disadvantages of serverless computing. (5)**
False: servers still exist, but the provider provisions, scales and manages them, so developers do not. Advantages: no server management; auto-scales to zero and pay only per invocation (also fast deployment). Disadvantages: cold-start latency; harder debugging.

**Q6. Differentiate between a virtual machine and a container. (4)**
A VM virtualizes the hardware and runs a full guest OS on a hypervisor — strong isolation, but GBs in size and minutes to boot. A container packages the app and dependencies and shares the host OS kernel through a container runtime — lighter (MBs), starts in seconds, more portable, weaker isolation.

**Q7. Compare edge, fog and cloud computing. Why does an autonomous vehicle need edge computing? (6)**
Cloud: centralized data centres, high latency (50–150 ms), massive compute, batch analytics. Fog: between edge and cloud, medium latency (10–50 ms), aggregates edge data. Edge: on device/gateway, ultra-low latency (<10 ms), lightweight real-time compute at the source. A vehicle must brake within milliseconds; waiting for a round-trip to a far data centre is too slow and the connection may drop, so decisions are made at the edge.

**Q8. State three principles of Zero Trust security. (3)**
Verify explicitly (authenticate using identity, location, device health, workload); least privilege (minimum access only when needed); assume breach (design as if attackers are inside, minimise blast radius, segment access).

**Q9. Explain why virtualization is called the core enabler of cloud computing. (4)**
A hypervisor divides one physical server into many isolated VMs. This allows resource pooling and multi-tenancy (many customers on shared hardware), high utilisation (server consolidation), rapid provisioning (on-demand self-service, elasticity), and snapshotting/migration for resilience — all the properties the cloud depends on.

**Q10. Give four FinOps techniques for reducing cloud costs. (4)**
Right-sizing instances; reserved instances for predictable workloads (up to 72% saving); spot/preemptible VMs for fault-tolerant work (60–90% discount); auto-scaling down when idle; storage tiering; cost-allocation tags.

**Q11. Why do 87% of enterprises use a multi-cloud strategy? (5)**
To avoid vendor lock-in; to use best-of-breed services from each provider; for regulatory compliance (in-country providers); for cost arbitrage between providers; and for redundancy (fail over during one provider's outage).

**Q12. Explain the 5 V's of big data. (5)**
Volume (amount: 120+ ZB in 2024), Velocity (speed of arrival, real-time streams), Variety (structured, semi-structured, unstructured), Veracity (quality and accuracy), Value (insights and business decisions).

## 13. Common mistakes

| Mistake | Correct idea |
|---|---|
| Mixing service models and deployment models | Service model = **what** is provided (IaaS/PaaS/SaaS). Deployment model = **who owns/uses** it (public/private/hybrid/community/multi). |
| "In IaaS the provider manages the OS." | In IaaS **you** manage the OS and everything above it. |
| Hybrid = multi-cloud | Hybrid = private + public. Multi-cloud = several public providers. |
| "Serverless has no servers." | Servers exist; you just don't manage them. |
| Listing "scalability" or "security" as a NIST characteristic | The five are on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service. |
| "The provider is responsible for all security in the cloud." | Shared responsibility: the customer secures data, IAM, network config, apps. |
| Containers contain their own OS | They share the host OS kernel; VMs have a full guest OS. |
| Edge = fog | Edge is on/near the device (<10 ms); fog is a middle layer (10–50 ms) that aggregates edge data. |
| Spot instances for a critical always-on server | Spot can be reclaimed at any time; use it only for fault-tolerant work. |

## Cheat sheet

| Basics | |
|---|---|
| Definition | Computing as a **service, not a product**; metered over a network; on-demand, pay-as-you-go |
| Key concepts | Remotely hosted, ubiquitous access, commodified computing, multi-tenancy |
| 5 V's | Volume, Velocity, Variety, Veracity, Value |
| NIST 5 | On-demand self-service, Broad network access, Resource pooling, Rapid elasticity, Measured service |
| Why | Pay-as-you-go (no CapEx, lower TCO), instant scale, simplified IT, flexible, efficient, green |

| Model | You manage | Example |
|---|---|---|
| IaaS | OS ↑ (OS, middleware, runtime, data, apps) | EC2, Azure VMs, GCE |
| PaaS | App + data | Heroku, App Engine, Beanstalk |
| SaaS | Nothing (just use) | Gmail, Salesforce, M365, Zoom |
| XaaS | FaaS (Lambda), DBaaS (RDS), STaaS (S3), AIaaS (SageMaker), NaaS (SDN), SECaaS (Zscaler) | |

| Deployment | |
|---|---|
| Public | 3rd party, shared; cheap, scalable |
| Private | One org; security, control |
| Hybrid | Private + public, portable; flexible |
| Community | Orgs with shared concerns |
| Multi-cloud | Several public; no lock-in (87%) |

| Modern developments | |
|---|---|
| Containers | Share host kernel; light, fast, portable (Docker) |
| Kubernetes | Auto-scale, self-heal, load balance, rolling updates, service discovery, storage orchestration (EKS/AKS/GKE) |
| Serverless | Trigger → run → scale 0..∞ → pay per run. Cons: cold start, debugging |
| Edge / Fog / Cloud | <10 ms / 10–50 ms / 50–150 ms |
| Virtualization | Consolidation, isolation, rapid provisioning, snapshot & migration |

| Security, cost, ops | |
|---|---|
| Shared resp. | Provider: OF the cloud (physical, network HW, hypervisor). Customer: IN (data, IAM, network config, apps) |
| Zero Trust | Never trust, always verify: verify explicitly, least privilege, assume breach |
| FinOps | Right-size, reserved (≤72%), spot (60–90%), auto-scale, tiering, tags |
| Compliance | GDPR, HIPAA, PCI-DSS, ISO 27001, SOC 2, PDPA |
| Multi-cloud why | Lock-in, best-of-breed, compliance, cost arbitrage, redundancy |
| Cloud-native | Microservices, CI/CD, IaC, observability, chaos eng., GitOps |
| Market | AWS 31%, Azure 25%, GCP 11%, Alibaba 4% |
| Green | DCs 1–2% of world power; cloud 65–80% more efficient |